

AMRITA VISHWA VIDYAPEETHAM

Amrita School of Engineering, Chennai

Department of Electronics and Communication Engineering

AIZEN ALGORITHM REPORT

OPERATING SYSTEMS CAPSTONE PROJECT

AIZEN: A Contextual-Bandit Algorithm for Routing Processes Between Performance and Efficiency Core Groups in User Space

Author	Role	Roll Number	Programme
Priyanraj	Main author	CH.EN.U4CCE25020	B.Tech CCE
Naveen	Co-author	CH.EN.U4CCE25014	B.Tech CCE

Submitted to: Dr. Dasari Naga Vinod

Submission deadline: 10 October 2026

October 2026

TABLE OF CONTENTS

List of Abbreviations.....	v
Abstract.....	vi
1. INTRODUCTION.....	1
1.1 Background and Motivation.....	1
1.2 Problem Statement.....	1
1.3 Objectives.....	1
1.4 Research Questions.....	1
1.5 Contributions.....	1
1.6 Code and Deployment Availability.....	2
1.7 Organisation of the Report.....	2
2. LITERATURE SURVEY.....	3
2.1 Heterogeneous Multicore Processors.....	3
2.2 General-Purpose Process Scheduling.....	3
2.3 Learning-Based Resource Management.....	3
2.4 Contextual Bandits.....	3
2.5 Tail Latency and Interactive Workloads.....	3
2.6 Summary and Research Gap.....	3
3. PROPOSED METHODOLOGY.....	5
3.1 System Overview.....	5
3.2 Contextual-Bandit Formulation.....	6
3.3 Reward Design.....	7
3.4 Control Loop.....	7
3.5 Safety and Robustness.....	8
3.6 Why a Contextual Bandit?.....	8
3.7 Context Features.....	9
3.8 Reward Landscape.....	10
3.9 Hyperparameters and Complexity.....	10
4. IMPLEMENTATION.....	11
4.1 Software Stack and Code Organisation.....	11
4.2 Telemetry and Actuation.....	11
4.3 Workload Generator.....	11
4.4 Dashboard.....	11
4.5 Verification of the Prototype.....	11
4.6 Heterogeneous-Core Simulator.....	12
4.7 Reproduction.....	13
4.8 Deployment and Availability.....	13
5. RESULTS AND ANALYSIS.....	14
5.1 Experimental Setup and Metrics.....	14
5.2 Overall Performance.....	14
5.3 Learning Dynamics.....	16
5.4 Statistical Analysis.....	17

5.5 Latency, Fairness and Migration.....	18
5.6 Sensitivity and Overhead.....	18
5.7 Policy Behaviour and Interpretability.....	19
5.8 Distribution Across Seeds and Steady State.....	20
5.9 Answers to the Research Questions.....	22
5.10 Discussion and Threats to Validity.....	22
6. CONCLUSION.....	23
APPENDIX A: KEY CODE LISTINGS.....	24
A.1 LinUCB with Sherman–Morrison update.....	24
A.2 Reward functions.....	24
A.3 Contention model.....	24
APPENDIX B: REPRODUCTION INSTRUCTIONS.....	25
APPENDIX C: SUPPLEMENTARY RESULTS.....	26
REFERENCES.....	27

TABLE OF FIGURES & TABLES

List of Figures

Fig. 1. Architecture of the AIZEN governor: telemetry feeds a context builder; the shared Lin.....	5
Fig. 2. Machine-learning workflow: (A) the online learning loop executed in every scheduling	6
Fig. 3. Structure of the AIZEN model: two linear reward heads with uncertainty bonuses, combi.....	7
Fig. 4. Flowchart of one scheduling cycle of the governor.....	8
Fig. 5. Comparison of learning paradigms for core-group selection; the contextual bandit used.....	9
Fig. 6. Illustration of upper-confidence-bound exploration on synthetic data: (a) after 5 sam.....	9
Fig. 7. Expected reward of each core group by process type: (a) proposed reward with $\lambda = 0.5$;.....	10
Fig. 8. Normalised throughput (a) and average power (b) of the nine policies (mean and 95% CI....	15
Fig. 9. Throughput–power trade-off; points show the mean over 30 seeds with 95% CI bars.....	15
Fig. 10. Placement over time (mean $\pm$ 1 s.d. over 30 seeds): (a) fraction of CPU-bound processe....	16
Fig. 11. Normalised throughput per scheduling cycle (10-cycle moving average, mean over 20 see...	16
Fig. 12. Cumulative throughput shortfall relative to Best-Static (mean over 20 seeds).....	17
Fig. 13. Interactive tail latency (a), Jain’s fairness index (b) and migration rate (c) of the.....	18
Fig. 14. Sensitivity of LinUCB-Perf+IPC (15 seeds per point) to the energy weight λ (a) and th.....	18
Fig. 15. Fraction of time each process type spends on the P-group under each policy (mean over....	19
Fig. 16. Weights learned by the shared LinUCB-Perf+IPC bandit for each arm at the end of one 3... 20	20
Fig. 17. Distribution over 30 seeds of normalised throughput (a) and efficiency (b); boxes sho.....	20
Fig. 18. Normalised throughput (a) and efficiency (b) over all 300 cycles versus the last 100	21
Fig. 19. Mean value of every metric for every policy; shading indicates relative rank within e.....	21
Fig. 20. Radar chart of seven normalised metrics for five policies (outer ring = best policy o.....	22

List of Tables

Table I. Comparison of Related Approaches.....	4
Table II. Context Features.....	9
Table III. Hyperparameters and Defaults.....	10
Table IV. Main Components of the Code Base.....	11
Table V. Findings of the Prototype Audit.....	12
Table VI. Simulator Parameters.....	12
Table VII. Simulated Workload Mix (12 Processes).....	13
Table VIII. Metric Definitions.....	14
Table IX. Evaluated Policies.....	14
Table X. Mean $\pm$ 95% CI over 30 Seeds.....	14
Table XI. Paired Differences (Holm-Adjusted Wilcoxon).....	17
Table XII. Computational Cost of One Bandit Update ($d = 6$).....	19
Table XIII. Steady-State and Supplementary Results.....	26

LIST OF ABBREVIATIONS

Abbreviation	Meaning
P-core / E-core	Performance core / Efficiency core
CFS	Completely Fair Scheduler
EAS	Energy Aware Scheduling
UCB	Upper confidence bound
LinUCB	Linear contextual bandit with UCB exploration
SM	Sherman–Morrison rank-one inverse update
IPC	Instructions per cycle
RQ	Research question
CI	Confidence interval
d _z	Paired Cohen’s effect size
PID	Process identifier
RAPL	Running Average Power Limit (Intel energy counters)
p95	95th percentile

ABSTRACT

Modern processors increasingly combine fast, power-hungry Performance (P) cores with slower, energy-frugal Efficiency (E) cores, yet a general-purpose scheduler must decide where each process should run with little knowledge of what it will do next. This report presents AIZEN, a user-space governor that treats the choice between the P-core group and the E-core group as a contextual-bandit problem solved with LinUCB. In every one-second cycle the governor builds a context vector for each process from operating-system telemetry (CPU share, voluntary and involuntary context-switch rates, current placement and, optionally, an IPC estimate), selects a core group with an upper-confidence-bound rule, applies it through the CPU-affinity interface, and learns from a measured reward. We audit the original prototype, show that a reward made of involuntary context switches and a constant energy proxy drives the policy toward low power at the expense of throughput, and replace it with a service-minus-energy reward. Because the development machine has neither a hybrid CPU nor trustworthy involuntary-switch counters, the quantitative evaluation uses a calibrated discrete-time simulator of a 4P+4E processor running twelve processes for 300 cycles over 30 seeds. The proposed policy (AIZEN with the IPC feature, labelled LinUCB-Perf+IPC in the tables) reaches 0.827 normalised throughput, 46% above the README heuristic (0.565), 13% above random placement (0.731) and within 5% of a hindsight-optimal static partition (0.870) that must know every process type in advance, at 5.30 relative power units. Adding an IPC feature reduces migrations from 12.1 to 2.4 per process per minute. The study also exposes a limitation: the tail latency of interactive processes is higher (13.2 ms versus about 2 ms for label-based routing), which motivates a latency-aware reward. Code, raw per-seed data and figures are provided for reproduction.

Index Terms—heterogeneous multicore, CPU scheduling, contextual bandit, LinUCB, energy efficiency, operating systems.

1. INTRODUCTION

1.1 Background and Motivation

Processor design has moved from many identical cores to heterogeneous multicore chips in which a few fast, power-hungry cores coexist with many slower, energy-frugal ones. The idea was demonstrated architecturally by Kumar et al. [1], who showed that moving programs between cores of different capability can reduce processor power with limited performance loss, and it reached mass-market devices through ARM big.LITTLE [2] and, more recently, hybrid x86 processors with Performance (P) and Efficiency (E) cores. The operating system is responsible for deciding which process runs on which core class, and mainstream kernels address this with fair-share scheduling [3] extended by an energy model for asymmetric CPUs [4].

Static rules such as “CPU-heavy work goes to the P-cores, everything else to the E-cores” are easy to implement but cannot follow workloads whose behaviour changes over time, and they ignore contention: when twice as many CPU-bound processes exist as P-cores, a rule that keeps all of them on the P-group leaves the E-group idle. Scheduler research has also shown that even mature kernels can leave cores idle while runnable threads wait [5]. A policy that learns from the measured outcome of its own placements is therefore attractive, provided it is cheap enough to run continuously and safe enough to deploy without kernel changes.

1.2 Problem Statement

Given a set of running processes and two core groups with different speed and power, decide at every scheduling cycle which group each process should use so that the system delivers as much useful work as possible per unit of energy, without prior labels of what each process is, without root privileges, and without modifying the kernel. The decision must adapt when a process changes phase (for example, from I/O waiting to computation).

1.3 Objectives

1. Formulate P/E core-group selection as a contextual-bandit problem that can be solved online in user space.
2. Audit the original AI-DAX Lite prototype and correct its reward signal and evaluation methodology.
3. Compare the learned policy with simple, heuristic and hindsight-optimal static baselines under a reproducible protocol.
4. Quantify the contribution of individual design choices (reward, IPC feature, exploration and energy weights) and the computational overhead.

1.4 Research Questions

The study is organised around four research questions.

- **RQ1.** Does the reward of the original prototype lead the governor to a useful placement policy?
- **RQ2.** How close can an online learner come to a hindsight-optimal static partition, and how does it compare with simple rules and random placement?
- **RQ3.** Which design choices (reward, IPC feature, energy weight λ , exploration coefficient α) matter most?
- **RQ4.** What does the policy cost in migrations, interactive tail latency and computation?

1.5 Contributions

- A LinUCB-based governor that routes processes between core groups using the sched_setaffinity interface and operating-system telemetry only.

- Identification of a failure mode of the original reward (involuntary context switches plus a constant energy proxy) and a replacement reward that combines delivered work and attributed energy.
- An ablation showing that a memory-stall/IPC feature makes memory-bound and CPU-bound processes separable and reduces unnecessary migrations by about 80%.
- A seeded, open simulator, a hindsight-optimal static baseline, and a statistical protocol (paired Wilcoxon tests with Holm correction) that make the results reproducible.
- A FastAPI/WebSocket dashboard and safety mechanisms (PID-reuse guard, atomic logging, affinity reset) for live demonstration.

1.6 Code and Deployment Availability

AIZEN is the name used in this report for the improved algorithm and governor built on the AI-DAX Lite prototype of the repository. The source code is available at <https://github.com/priyanraj-hub/operatingsystemsprojectcapstone> and the dashboard front end is deployed on Vercel under the project name “operatingsystemsprojectcapstone”. The simulator, figure scripts and raw per-seed data used in this report accompany the submission (Appendix B).

1.7 Organisation of the Report

Section 2 reviews related work. Section 3 presents the proposed methodology. Section 4 describes the implementation, the verification of the prototype, and the simulator. Section 5 reports and analyses the results. Section 6 concludes and lists future work. Appendices A–C give code listings, reproduction instructions and supplementary tables.

2. LITERATURE SURVEY

2.1 *Heterogeneous Multicore Processors*

Kumar et al. [1] introduced single-ISA heterogeneous multicore processors and showed, using a thread-to-core assignment that follows program phases, that significant power reductions are possible at modest performance cost. ARM’s big.LITTLE pairs high-performance and high-efficiency cores that share an instruction set [2]. In Linux, Energy Aware Scheduling uses an energy model of the CPU topology to place waking tasks on the core that minimises estimated energy while keeping the CPU capacity sufficient [4]. These mechanisms live inside the kernel and rely on platform-supplied energy models.

2.2 *General-Purpose Process Scheduling*

The Completely Fair Scheduler allocates CPU time in proportion to weights using virtual runtime ordering [3], and operating-systems textbooks treat it as the canonical fair-share design [6]. Lozi et al. documented performance bugs in the Linux scheduler that left cores idle while threads were waiting [5], which illustrates that placement decisions remain difficult even without heterogeneity. Fairness is commonly summarised with Jain’s index [7], which we also use.

2.3 *Learning-Based Resource Management*

Quasar uses classification techniques to estimate the resource needs and interference sensitivity of cluster workloads so that quality-of-service targets are met with fewer resources [8]. Decima applies deep reinforcement learning with graph neural networks to schedule data-processing jobs on clusters [9]. Reinforcement learning in general targets sequential decision making with delayed reward [10]; deep variants reached human-level performance in Atari games [11]. Both systems operate at the cluster level; neither addresses per-process core-class selection on a single host.

2.4 *Contextual Bandits*

When the decision at each step has immediate, measurable feedback and the action set is small, a contextual bandit is a lighter alternative to full reinforcement learning. Auer analysed linear upper-confidence-bound exploration [12]; LinUCB, introduced by Li et al. for news recommendation, maintains a ridge-regression estimate [13] per arm and adds an exploration bonus proportional to the uncertainty of the prediction [14]. Regret analyses for linear payoffs and improved confidence sets are given in [15], [16], and a comprehensive treatment is available in [17]. LinUCB needs only a $d \times d$ matrix per arm, which makes it suitable for continuous in-process use.

2.5 *Tail Latency and Interactive Workloads*

For interactive services the high percentiles of latency matter more than the mean, because rare slow responses dominate the experience of the user when many requests are involved [18]. Placing an interactive process on a slower core class or on a crowded core therefore trades energy for tail latency, which is why this report measures the p95 latency of interactive processes separately from throughput.

2.6 *Summary and Research Gap*

Table I contrasts the surveyed approaches. Kernel mechanisms are hardware-aware but not adaptive to measured outcomes; cluster-level learners are adaptive but operate at a different granularity. What is missing, and what this work explores, is a lightweight, online-learning governor that works at process granularity from user space and whose behaviour can be inspected and reproduced by students and practitioners.

TABLE I
COMPARISON OF RELATED APPROACHES

Work	Layer	Method	Learns online	Relation to this work
Kumar et al. [1]	Architecture	Phase-based thread-to-core migration	No	Motivates P/E routing
big.LITTLE [2]	Hardware/OS	Asymmetric core pairing	No	Target platform class
Linux EAS [4]	Kernel	Energy-model task placement	No	Kernel-level counterpart
CFS [3]	Kernel	Virtual-runtime fair sharing	No	Default baseline
Quasar [8]	Cluster	Classification of resource needs	Partly	Different granularity
Decima [9]	Cluster	Deep RL job scheduling	Yes	Heavier, cluster level
LinUCB [14]	Algorithm	Linear contextual bandit	Yes	Algorithm used here

3. PROPOSED METHODOLOGY

3.1 System Overview

AIZEN runs as an ordinary user-space process. Each cycle it reads telemetry of every tracked process, builds a context vector, selects a core group with a LinUCB policy, applies the choice by setting the process's CPU-affinity mask, and learns from the outcome measured in the next cycle. Fig. 1 shows the architecture and Fig. 4 the control loop.

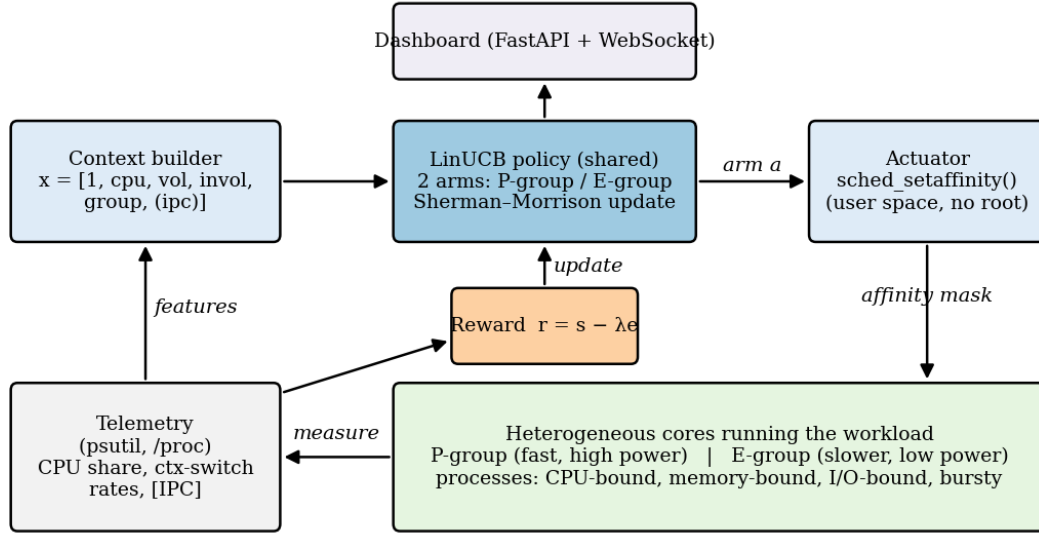


Fig. 1. Architecture of the AIZEN governor: telemetry feeds a context builder; the shared LinUCB policy selects a core group; the actuator applies the affinity mask; the measured outcome yields the reward that updates the policy.

The machine-learning workflow of AIZEN has two parts (Fig. 2). The online loop (A) runs once per scheduling cycle: telemetry is turned into a scaled feature vector, the two per-arm models score the arms with an upper confidence bound, the chosen core group is applied, and the outcome measured in the next cycle becomes a reward that updates the model in $O(d^2)$. The offline pipeline (B) is how the policies are evaluated: a seeded workload runs on the simulated 4P+4E machine under nine policies and 30 seeds, the metrics are summarised, tested statistically and turned into the figures and tables of Section 5.

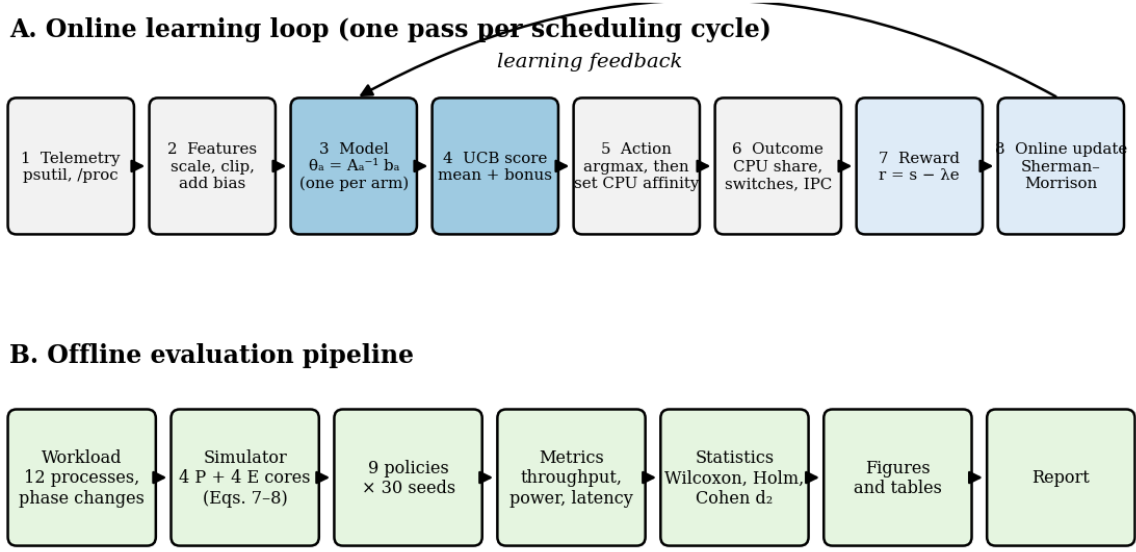


Fig. 2. Machine-learning workflow: (A) the online learning loop executed in every scheduling cycle; (B) the offline evaluation pipeline used to produce the results of this report.

3.2 Contextual-Bandit Formulation

For process i at cycle t the context vector is

$$\mathbf{x} = [1, c, v/300, \min(n/100, 3), g, q]^T \quad (1)$$

where c is the CPU share obtained in the previous cycle, v and n are the voluntary and involuntary context-switch rates per second, $g \in \{0, 1\}$ is the current group ($0 = P$, $1 = E$), and q is an optional IPC estimate. The constant 1 provides an intercept. There are two arms, $a \in \{P, E\}$. For each arm the policy keeps a matrix A and a vector b (initialised to the identity and zero) and scores

$$\text{score}(a) = \theta_a^T \mathbf{x} + \alpha \cdot \sqrt{\mathbf{x}^T A_a^{-1} \mathbf{x}}, \quad \theta_a = A_a^{-1} b_a \quad (2)$$

and picks the arm with the larger score; ties are broken at random. After the reward r of the previous action is observed the chosen arm is updated:

$$A_a \leftarrow A_a + \mathbf{x} \mathbf{x}^T, \quad b_a \leftarrow b_a + r \mathbf{x} \quad (3)$$

We maintain A^{-1} directly with the Sherman–Morrison identity [19],

$$A^{-1} \leftarrow A^{-1} - (A^{-1} \mathbf{x} \mathbf{x}^T A^{-1}) / (1 + \mathbf{x}^T A^{-1} \mathbf{x}) \quad (4)$$

which costs $O(d^2)$ per update instead of $O(d^3)$ for explicit inversion. One bandit is shared by all processes, so experience from one process informs the others; the cost of sharing is that processes interact through contention, which makes the reward non-stationary (Section 5.10).

Fig. 3 shows the structure of the model. Six inputs feed two linear heads, one per core group. Each head is a ridge-regression predictor of the reward of its arm together with a covariance matrix that records how well the arm has been explored; the upper confidence bound of an arm is the sum of its predicted reward and an uncertainty bonus, and the arm with the larger bound is chosen. The model has $2 \times 6 = 12$ weights and two 6×6 covariance matrices and no hidden layers, so every weight can be read directly (Section 5.7) and the whole model is updated in microseconds.

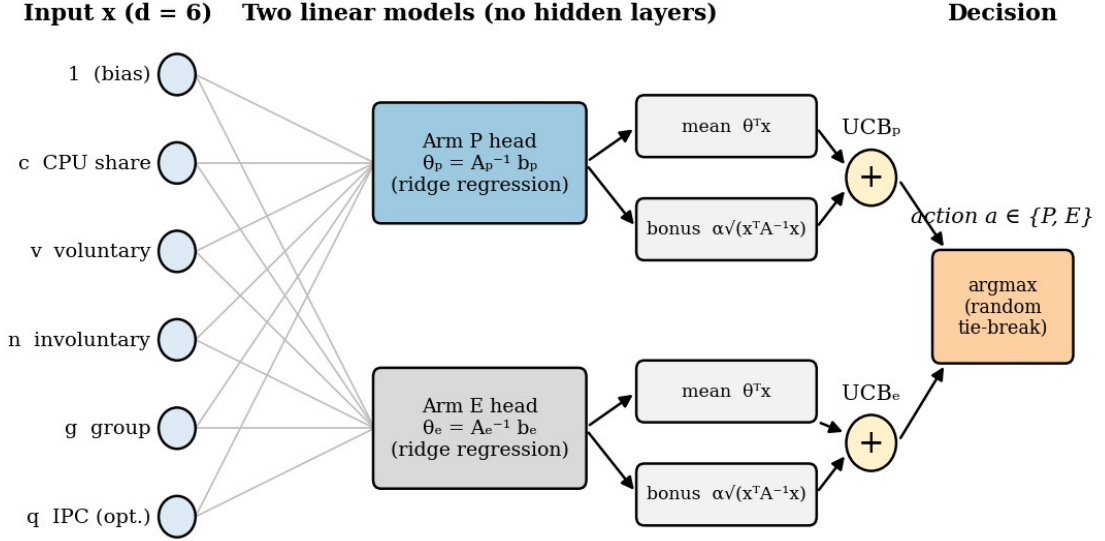


Fig. 3. Structure of the AIZEN model: two linear reward heads with uncertainty bonuses, combined into upper confidence bounds and a greedy arm choice.

3.3 Reward Design

The original prototype used

$$r = -(0.7 \tilde{n} + 0.3 \varepsilon_a), \quad \varepsilon_P = 1.0, \varepsilon_E = 0.3 \quad (5)$$

where $\tilde{n}$ is a normalised involuntary-switch rate. The energy term depends only on the chosen arm, and $\tilde{n}$ is close to zero whenever there is no contention, so the reward reduces to a constant preference for the E-group irrespective of the work being delivered. We therefore replace it with a service-and-energy reward,

$$r = s - \lambda e, \quad s = \text{delivered work} / \text{demanded work}, \quad e = c \cdot \pi_g \quad (6)$$

where s is the fraction of the process's demand that was actually served (measurable from retired-instruction counters or, as a proxy, CPU share), e is the CPU share multiplied by the dynamic power π of the group used, and $\lambda = 0.5$ is the energy weight. In an uncontended system this gives, for a CPU-bound process, $r = 1 - 0.5 = 0.50$ on a P-core versus $0.55 - 0.15 = 0.40$ on an E-core, so P is preferred; for a memory-bound process (demand 0.8, E-core speed 0.9) it gives 0.60 on P versus 0.78 on E, so E is preferred. Under contention the served fraction drops on an overloaded group, which pushes processes toward the less loaded group and produces load balancing without any process labels.

3.4 Control Loop

Algorithm 1 summarises one scheduling cycle and Fig. 4 shows the same loop as a flowchart. The learning step uses the outcome of the previous cycle, so every update is based on a measured effect of an earlier placement rather than on a heuristic label.

Algorithm 1: AIZEN scheduling cycle

- 1: for each tracked process i (validated by PID and creation time):
 - 2: read telemetry: $c, v, n, (q)$ over the last cycle
 - 3: build context x as in (1)
 - 4: if a previous action a' exists for i :
 - 5: compute reward $r = s - \lambda e$ as in (6)
 - 6: update A^{-1}, b of arm a' with (3) and (4)
 - 7: compute $\text{score}(P), \text{score}(E)$ as in (2)
-

```

8:  $a \leftarrow \text{argmax score}$  (random tie-break)
9: if  $a \neq \text{current group}$ : apply cpu_affinity(mask of a); count a migration
10: store  $(a, x)$  for the next cycle
11: log decisions, publish to dashboard, sleep until next cycle

```

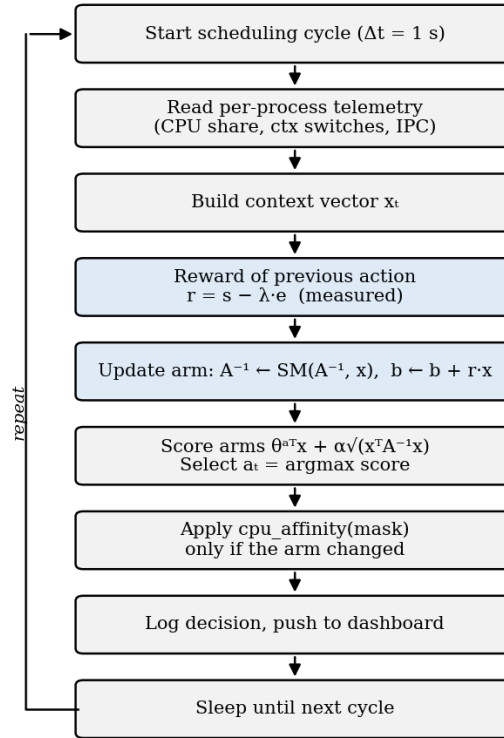


Fig. 4. Flowchart of one scheduling cycle of the governor.

3.5 Safety and Robustness

The governor never needs root privileges because it only changes the affinity of processes owned by the same user. It validates each tracked process by comparing the recorded creation time with the live process to avoid acting on a recycled PID; log files are written atomically through a temporary file and a rename; SIGINT and SIGTERM handlers restore the full CPU set for every tracked process; and the dashboard exposes an emergency-reset endpoint that releases all affinity constraints.

3.6 Why a Contextual Bandit?

Three learning paradigms could drive a placement policy (Fig. 5). Supervised learning requires a label saying which core group was best for each process, but no such ground truth exists because the outcome of the alternative placement is never observed. Full reinforcement learning models the effect of an action on future states and optimises a discounted return, which needs far more samples and heavier models [10], [11]. A contextual bandit sits between them: it learns from the reward of the action that was actually taken, needs no transition model, and its update is a few small matrix operations. Because a placement mostly affects the throughput and energy of the next cycle, ignoring long-term state is an acceptable approximation; the price is that interactions between processes are not modelled explicitly (Section 5.10).

Supervised learning	Contextual bandit (this work)	Full reinforcement learning
Labelled "best core" per process	Reward of the chosen action only	Delayed return over state sequences
No ground-truth labels exist	Immediate reward $r = s_{t+1} - s_t$	Needs a state-transition model / many samples
Offline training, fixed model	Online, $O(d^2)$ update per decision	Neural-network training and inference
Cannot adapt to phase changes	Adapts as rewards drift	Adapts, but slowly and at higher cost

Fig. 5. Comparison of learning paradigms for core-group selection; the contextual bandit used in this work (shaded) needs no labels and updates online.

Exploration is handled by the upper confidence bound. Fig. 6 illustrates the mechanism on synthetic data (it is an illustration, not an experimental result): with few samples the confidence band of each arm is wide, so the arm that currently looks worse may still be tried; as evidence accumulates the bands shrink and the policy settles on the arm with the higher predicted reward, here arm E for a low CPU share and arm P for a high CPU share.

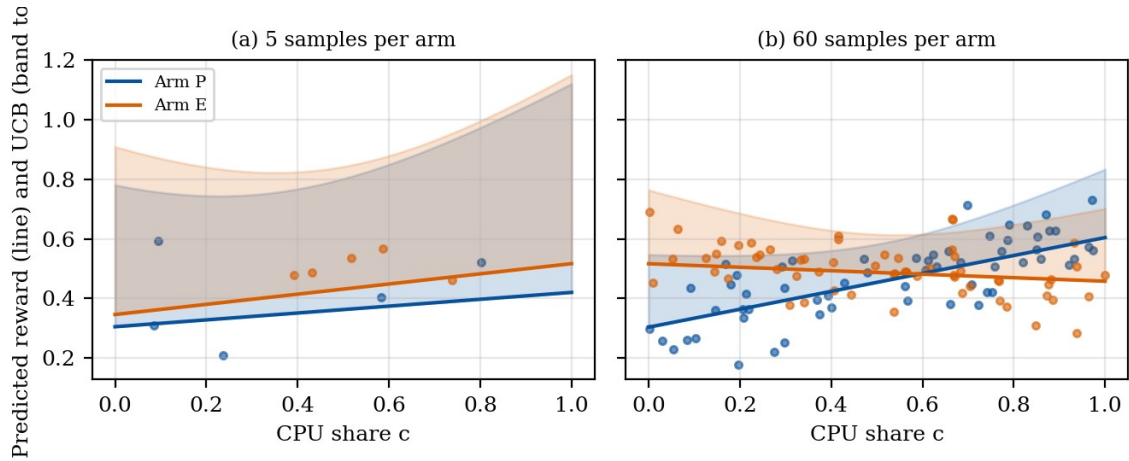


Fig. 6. Illustration of upper-confidence-bound exploration on synthetic data: (a) after 5 samples per arm the confidence bands are wide; (b) after 60 samples they have narrowed and the preferred arm depends on the CPU share. Lines are predicted rewards and the top of each band is the UCB.

3.7 Context Features

Table II lists the elements of the context vector in (1) and how they are obtained and scaled.

TABLE II
CONTEXT FEATURES

Feature	Source	Scaling	Purpose
Bias	constant 1	—	Intercept: baseline reward of an arm
CPU share c	CPU-time delta $\div \Delta t$ (psutil)	clipped to $[0, 1]$	Demand and service level
Voluntary switches v	/proc counter delta $\div \Delta t$	$\div 300$	I/O-boundness (sleeps often)
Involuntary switches n	/proc counter delta $\div \Delta t$	$\div 100$, clipped to $[0, 3]$	Contention on the current group
Group g	current affinity class	$0 = P, 1 = E$	Lets the model value staying put
IPC q (optional)	retired instructions $\div$ cycles (performance counters)	as measured	Separates memory-bound from compute-bound

Fixed scales are used instead of running statistics so that a stored context is never re-scaled by later observations. On a real system the IPC feature needs hardware performance counters, which psutil does not provide; in the simulator it stands for any counter-based memory-intensity signal such as stall cycles.

3.8 Reward Landscape

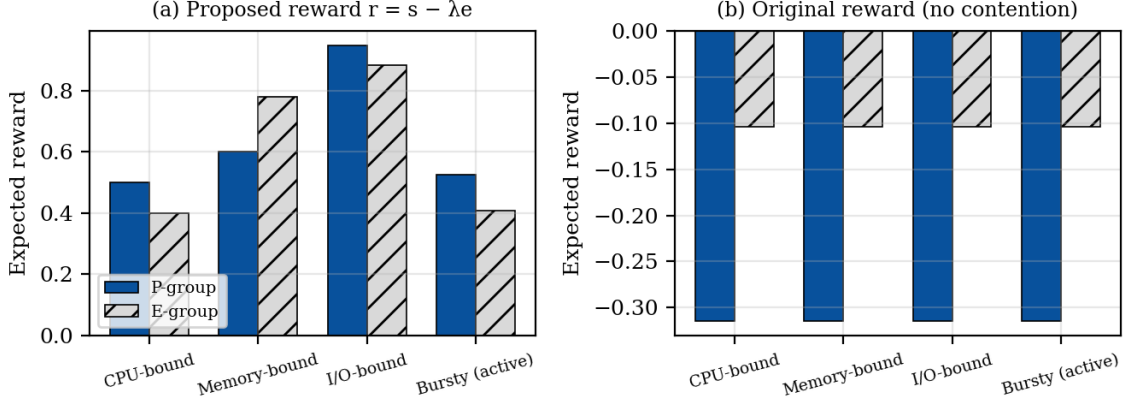


Fig. 7. Expected reward of each core group by process type: (a) proposed reward with $\lambda = 0.5$; (b) original reward without contention.

Fig. 7(a) plots the expected reward of (6) for $\lambda = 0.5$ in the absence of contention. CPU-bound and bursty-active processes obtain 0.50 and about 0.53 on the P-group but only 0.40 and 0.41 on the E-group; memory-bound processes obtain 0.78 on E against 0.60 on P; I/O-bound processes are almost indifferent (0.95 against 0.89). The learned policy therefore only has to discover the sign of these differences. Fig. 7(b) shows the original reward in the same situation: with $\tilde{n} \approx 0.02$ it gives -0.31 on P and -0.10 on E for every process type, so the E-group is always preferred and nothing rewards giving a process the speed it needs.

3.9 Hyperparameters and Complexity

TABLE III
HYPERPARAMETERS AND DEFAULTS

Parameter	Default	Range / note
Exploration coefficient α	0.5	swept 0.05–2.0
Energy weight λ	0.5	swept 0.1–1.2
Ridge regulariser	1.0 (A initialised to I)	fixed
Cycle length Δt	1 s	fixed
Arms	2 (P-group, E-group)	fixed
Feature dimension d	5 (6 with IPC)	fixed
Tie-breaking	uniform random	fixed
Bandit sharing	one bandit for all processes	per-PID bandit left for future work

Each decision costs two matrix–vector products and one quadratic form per arm, $O(d^2)$; each update is one rank-one correction, also $O(d^2)$. Memory per bandit is $2(d^2 + d)$ numbers (84 for $d = 6$), independent of the number of decisions, and a single shared bandit serves all processes.

4. IMPLEMENTATION

4.1 Software Stack and Code Organisation

The prototype is written in Python 3. Process telemetry and affinity control use psutil [20], numerical work uses NumPy and SciPy, and the dashboard is a FastAPI application with a REST interface and a WebSocket stream consumed by an HTML/JavaScript front end. Table IV lists the main components.

TABLE IV
MAIN COMPONENTS OF THE CODE BASE

File	Role
ai_scheduler.py	LinUCB bandit, telemetry, reward, affinity actuation, bandit statistics
workload_generator.py	Spawns CPU-bound and I/O-bound worker processes (infinite or finite jobs)
evaluate.py	Four-condition native evaluation harness (CFS, static, heuristic, bandit)
main.py, static/	FastAPI REST/WebSocket backend and dashboard front end
sim.py (new)	Seeded heterogeneous-core simulator and all policies; writes results and raw data
figs.py (new)	Generates every figure used in this report from the simulator output

4.2 Telemetry and Actuation

For each tracked process the governor reads CPU times, voluntary and involuntary context-switch counters, and the current affinity mask; the counters are exposed through /proc [21]. Rates are computed as differences between consecutive cycles. A persistent process handle must be kept between cycles, because the CPU-percentage routine of psutil returns a meaningless zero on the first call of a new handle. Actuation uses the CPU-affinity call [22] with the mask of the P-group (lower half of the logical CPUs by default or the sysfs-reported P-cores where available) or the E-group.

4.3 Workload Generator

The generator starts a configurable mix of CPU-bound workers (prime computation) and I/O-bound workers (small file writes and reads separated by sleeps). Each process registers its PID, type label and creation time in a tracking file that the scheduler and the dashboard read. A finite mode makes jobs terminate so that completion time can be measured.

4.4 Dashboard

The dashboard shows per-process placement, reward and bandit statistics, system CPU health and a reset button. The backend reads the scheduler’s CSV log and statistics file once per second and pushes them over a WebSocket.

4.5 Verification of the Prototype

Before using the native harness for the evaluation we inspected its source and output and found four issues that make its numbers unsuitable as evidence: (i) the archived native results show zero involuntary context switches, zero heavy-task completion time and undefined p-values in every condition, so no effect can be measured from them; (ii) the bandit condition of the harness creates a new process handle in every cycle, so the CPU-share feature is read from a first-call handle; (iii) the heuristic-only condition measures metrics over a different, shorter window than the other conditions; and (iv) the static-naive condition reads each process’s ground-truth type from the tracking file, so it is an oracle rather than a naive rule. These findings are the reason the quantitative evaluation in this report is performed on a simulator with explicit assumptions, and they define the corrections needed for a native Linux re-run (Section 6).

Table V collects the findings of the audit in one place, with the evidence and the corrective action adopted in this report.

TABLE V
FINDINGS OF THE PROTOTYPE AUDIT

ID	Finding	Evidence	Consequence	Corrective action
A1	Archived native results are degenerate	final_results.txt: involuntary switches 0.00, heavy completion 0.00, p-values undefined in every condition	No effect can be measured or claimed	Re-run on Linux with oversubscribed cores; report all metrics
A2	Heavy workers appear unmeasured	Mean CPU of about 0.4–0.9% although CPU-bound workers were launched	Experiment effectively measured idle I/O workers	Verify worker liveness; log samples per process
A3	CPU share read from a fresh handle each cycle	Scheduler and harness create a new psutil handle per cycle; first call returns 0.0	CPU feature constant; heuristic degenerates to the E-group	Keep one handle per PID across cycles
A4	Heuristic condition measured over a different window	evaluate.py: control phase unmeasured; metrics taken afterwards for at most 10 s	Conditions not comparable	Identical measurement window for all conditions
A5	“Static-naive” baseline uses ground-truth labels	Type read from the tracking file	Oracle, not naive	Rename; add label-free baselines
A6	Reward and regret ill-posed	Energy term depends only on the arm; both arms scored with the same observed involuntary rate	Policy reduces to “prefer E”; regret not counterfactual	Reward (6); simulation-based true regret
A7	Dashboard hardening	Unauthenticated reset endpoint; README suggests binding to 0.0.0.0; log truncated to 300 lines	Exposure on shared networks; no raw data retained	Local-only binding, token, append-only raw log
A8	Repository hygiene	Temporary and log files committed; no tests; no standard licence file	Hard to reproduce and reuse	Clean tree, tests, CI, explicit licence

4.6 Heterogeneous-Core Simulator

The development machine has no hybrid CPU, so the evaluation uses a discrete-time simulator whose cycle length equals the governor’s one-second period. A machine has $n_P = 4$ P-cores and $n_E = 4$ E-cores. Process i has a demand d_i (fraction of one P-core it wants) and a relative speed σ_i on the E-group (1.0 on P). If D_g is the total demand placed on group g with n_g cores, each process receives

$$\text{share}_i = d_i \cdot \min(1, n_g / D_g), \quad \text{progress}_i = \text{share}_i \cdot \sigma_{i,g} \quad (7)$$

so an overloaded group serves only part of the demand. System power is

$$\text{Power} = \sum_g [n_g \pi_{\text{idle},g} + \pi_{\text{dyn},g} \cdot \min(n_g, D_g)] \quad (8)$$

Involuntary context switches follow $2d + 150 \cdot \text{share} \cdot (1 - n_g/D_g)$ per second and voluntary switches $10 + 300(1 - d)$, both with multiplicative log-normal noise ($\sigma = 0.2$). The tail latency of an interactive process is $(1/\sigma) / (1 - 0.9 \cdot \min(\rho_g, 1))$ milliseconds with log-normal noise ($\sigma = 0.3$), where $\rho_g = D_g / n_g$. Table VI lists the parameters and Table VII the workload mix, which is deliberately oversubscribed: the mean total demand is about 7.7 P-core equivalents against four P-cores. All values are modelling assumptions chosen to resemble a hybrid processor and are not measurements.

TABLE VI
SIMULATOR PARAMETERS

Parameter	Value
Cores	4 P-cores (speed 1.0), 4 E-cores
Dynamic power per busy core	$P = 1.00$, $E = 0.30$ (relative units)
Idle power per core	$P = 0.10$, $E = 0.03$
Cycle / episode	1 s / 300 cycles; first 200 cycles used for steady-state figures
Telemetry noise	log-normal, $\sigma = 0.2$ (counters), $\sigma = 0.3$ (latency)
Bandit	shared LinUCB, ridge 1, $\alpha = 0.5$, $\lambda = 0.5$, random tie-break
Seeds	30 independent seeds (0–29), common random numbers across policies

TABLE VII
SIMULATED WORKLOAD MIX (12 PROCESSES)

Type	Count	Demand	E-core speed	IPC feature
CPU-bound	4	1.00	0.55	1.0
Memory-bound	3	0.80	0.90	0.3
I/O-bound (interactive)	3	0.10	0.90	0.6
Bursty (phase-changing)	2	0.95 / 0.10 (20–40 s phases)	0.55 / 0.90	1.0 / 0.6

4.7 Reproduction

The experiment is reproduced with “python sim.py” (about three minutes on one core; writes results.json and raw_per_seed.csv) followed by “python figs.py”. Every number and figure in Section 5 is produced by these two scripts; the seeds are fixed.

4.8 Deployment and Availability

The source repository is <https://github.com/priyanraj-hub/operatingsystemsprojectcapstone> and the dashboard front end is also deployed on Vercel (project “operatingsystemsprojectcapstone”). Because the governor changes CPU affinity through operating-system calls, it only affects processes on the machine where it runs; a hosted page cannot govern a remote host, so the Python backend must be started locally for live operation and the hosted page serves as a demonstration of the interface. The simulator, the figure scripts and the raw per-seed data used in this report accompany the submission so that every number can be regenerated.

5. RESULTS AND ANALYSIS

5.1 Experimental Setup and Metrics

Nine policies are compared (Table IX). Best-Static is a strong reference: it exhaustively searches all static assignments (using the symmetry between processes of the same type) for the one that maximises the same service-minus-energy objective on sampled time steps, so it uses knowledge of the workload that the online policies do not have. Metrics per episode (defined in Table VIII) are normalised throughput (progress divided by demand), average power, efficiency (work per unit power), p95 latency of the interactive I/O processes, Jain’s fairness index [7] of per-process satisfaction, and migrations per process per minute. Results are means over 30 seeds with 95% confidence intervals. Differences are tested with paired Wilcoxon signed-rank tests [23], Holm-corrected [24] across the five planned comparisons, and effect sizes are reported as paired Cohen’s d_z [25].

TABLE VIII
METRIC DEFINITIONS

Metric	Definition	Better
Normalised throughput	$\Sigma \text{ progress} \div \Sigma \text{ demand over the episode}$	higher
Average power	Mean over cycles of (8)	lower
Efficiency	Mean total progress per cycle $\div$ mean power	higher
p95 latency	95th percentile of the latency of interactive (I/O-bound) processes	lower
Jain’s index	$(\Sigma s)^2 \div (N \Sigma s^2)$ of per-process satisfaction s	higher
Migrations	Group changes per process per minute	lower
Involuntary switches	Mean per process per second	lower

TABLE IX
EVALUATED POLICIES

Policy	Description	Knowledge used
Always-P / Always-E	All processes on one group	None
Random	Re-draw each process’s group every 10 cycles	None
Static-Label	Heavy types to P, I/O to E (README rule)	True process type
Best-Static	Best fixed assignment found by exhaustive search	Types, demand profile (oracle)
Heuristic	P if CPU share > 20%, else E (README cold-start rule)	CPU share
LinUCB-Orig	LinUCB with the original reward (5)	psutil telemetry
LinUCB-Perf	AIZEN without IPC: LinUCB with reward (6)	psutil telemetry
LinUCB-Perf+IPC	AIZEN (proposed): LinUCB with reward (6) and the IPC feature	Telemetry + IPC

5.2 Overall Performance

TABLE X
MEAN $\pm$ 95% CI OVER 30 SEEDS

Policy	Throughput	Power	Efficiency	p95 lat. (ms)	Jain	Migr./proc/min
Always-P	0.516 $\pm$ 0.001	4.52 $\pm$ 0.00	0.885 $\pm$ 0.000	16.33 $\pm$ 0.12	1.000 $\pm$ 0.000	0.00 $\pm$ 0.00
Always-E	0.349 $\pm$ 0.001	1.72 $\pm$ 0.00	1.575 $\pm$ 0.001	18.14 $\pm$ 0.13	0.946 $\pm$ 0.000	0.00 $\pm$ 0.00
Random	0.731 $\pm$ 0.006	4.90 $\pm$ 0.04	1.157 $\pm$ 0.006	15.58 $\pm$ 0.21	0.985 $\pm$ 0.001	3.01 $\pm$ 0.06
Static-Label	0.551 $\pm$ 0.001	4.61 $\pm$ 0.00	0.926 $\pm$ 0.000	1.95 $\pm$ 0.01	0.940 $\pm$ 0.001	0.00 $\pm$ 0.00
Best-Static	0.870 $\pm$ 0.006	5.43 $\pm$ 0.03	1.242 $\pm$ 0.003	12.12 $\pm$ 0.69	0.980 $\pm$ 0.003	0.00 $\pm$ 0.00
Heuristic	0.565 $\pm$ 0.001	4.65 $\pm$ 0.00	0.942 $\pm$ 0.000	2.05 $\pm$ 0.02	0.947 $\pm$ 0.001	0.42 $\pm$ 0.01
LinUCB-Orig	0.588 $\pm$ 0.026	3.44 $\pm$ 0.18	1.334 $\pm$ 0.020	17.79 $\pm$ 0.14	0.989 $\pm$ 0.004	13.37 $\pm$ 1.43
LinUCB-Perf	0.758 $\pm$ 0.020	5.00 $\pm$ 0.07	1.176 $\pm$ 0.019	14.56 $\pm$ 0.34	0.985 $\pm$ 0.002	12.09 $\pm$ 3.77

Policy	Throughput	Power	Efficiency	p95 lat. (ms)	Jain	Migr./proc/min
LinUCB-Perf+IPC	0.827 ± 0.006	5.30 ± 0.02	1.210 ± 0.005	13.20 ± 0.97	0.997 ± 0.000	2.38 ± 0.57

Table X and Fig. 8 show the main outcome. LinUCB-Perf+IPC delivers 0.827 normalised throughput, 46% more than the README heuristic (0.565), 13% more than random placement (0.731) and 9% more than the same bandit without the IPC feature (0.758). It reaches 95% of the throughput of the hindsight-optimal Best-Static partition (0.870) at 2.4% lower power (5.30 versus 5.43). The label-based and heuristic rules deliver only 0.55–0.57 because they keep the E-group almost idle while the P-group is oversubscribed, which is exactly the contention situation that motivates learning.

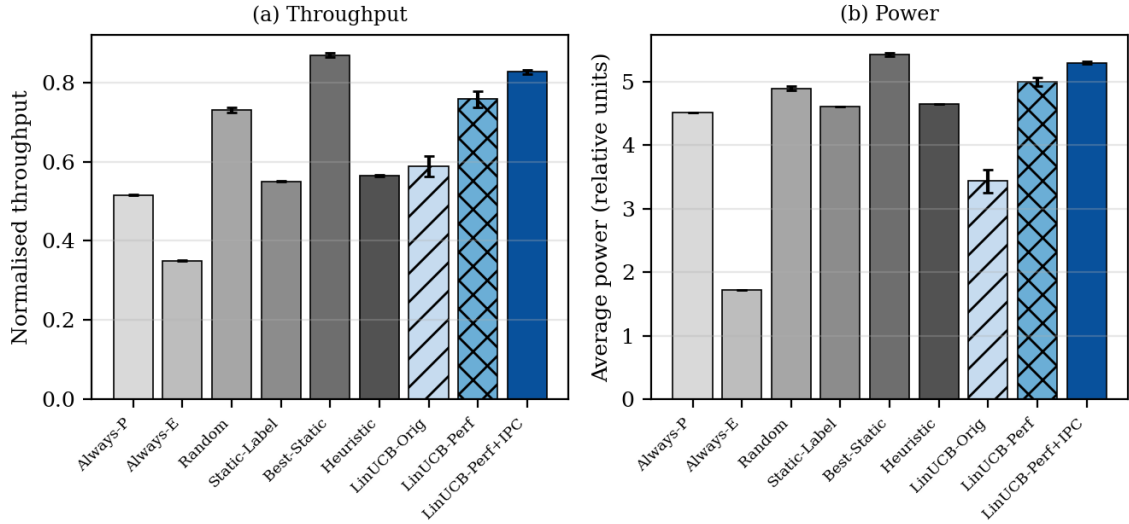


Fig. 8. Normalised throughput (a) and average power (b) of the nine policies (mean and 95% CI over 30 seeds).

Efficiency taken alone is misleading: Always-E has the highest efficiency (1.575) but delivers only 0.349 of the demanded work. Fig. 9 therefore shows the throughput–power trade-off. LinUCB-Perf+IPC lies on the upper-right frontier together with Best-Static, whereas LinUCB-Orig sits at low power and low throughput: with the original reward the bandit saves 31% power relative to LinUCB-Perf but loses 22% of the throughput, confirming the failure mode analysed in Section 3.3.

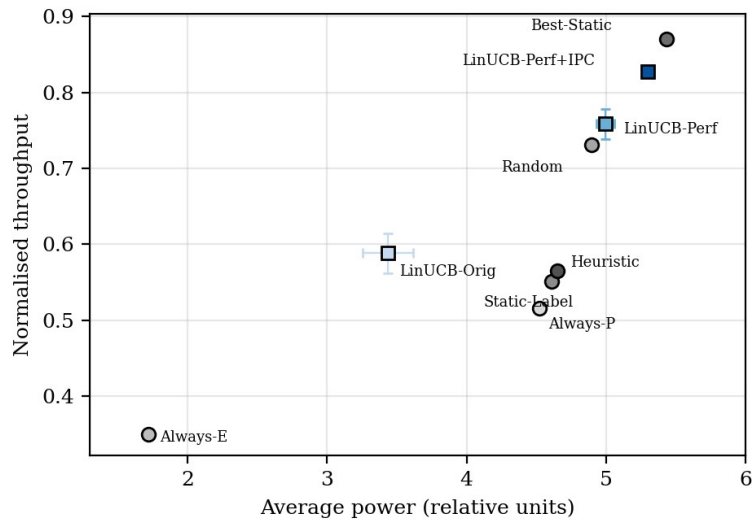


Fig. 9. Throughput–power trade-off; points show the mean over 30 seeds with 95% CI bars.

5.3 Learning Dynamics

Fig. 10 shows how placements evolve. With the IPC feature the policy places memory-bound processes on the E-group almost immediately (fraction near 1.0 within about ten cycles) and keeps roughly 90% of the CPU-bound processes on the P-group after about thirty cycles; the remaining share moves to the E-group when the P-group is saturated, which is the load-balancing behaviour of Section 3.3. Without the IPC feature the policy cannot tell memory-bound from CPU-bound processes using CPU share and context-switch rates alone, so only about 60–70% of memory-bound processes end up on the E-group and placements keep oscillating (12.1 migrations per process per minute versus 2.4 with IPC). The original reward keeps fewer than half of the CPU-bound processes on the P-group, in line with its low throughput.

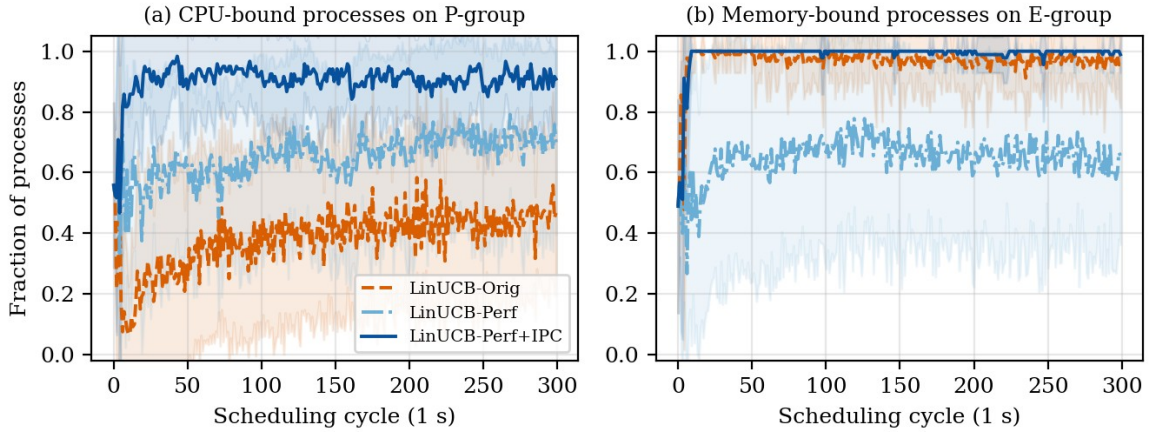


Fig. 10. Placement over time (mean $\pm$ 1 s.d. over 30 seeds): (a) fraction of CPU-bound processes on the P-group; (b) fraction of memory-bound processes on the E-group.

Fig. 11 shows the normalised throughput per cycle (10-cycle moving average, mean over 20 seeds) and Fig. 12 the cumulative throughput shortfall relative to Best-Static. This shortfall is a pseudo-regret, not the formal regret of the bandit, but it shows whether learning closes the gap. With IPC, the shortfall of AIZEN grows by 0.054 per cycle in cycles 1–100 and by 0.034 per cycle in cycles 200–300. The corresponding figures are 0.152 and 0.091 without IPC, 0.329 and 0.224 for the original reward, and 0.304 and 0.304 for the README heuristic, which does not learn.

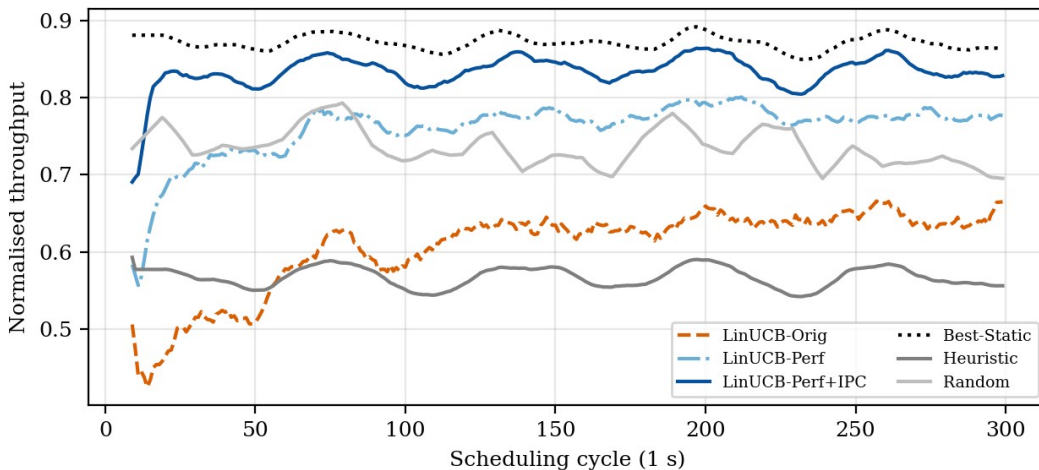


Fig. 11. Normalised throughput per scheduling cycle (10-cycle moving average, mean over 20 seeds).

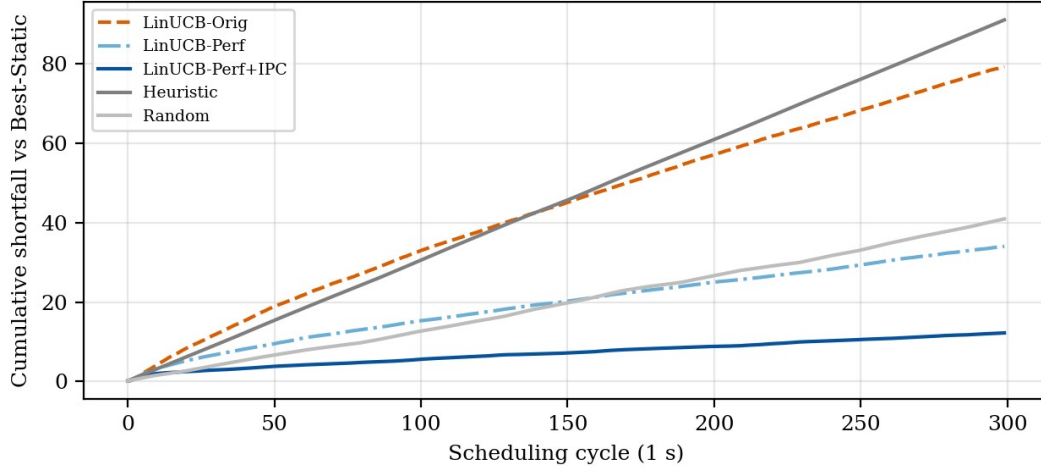


Fig. 12. Cumulative throughput shortfall relative to Best-Static (mean over 20 seeds).

5.4 Statistical Analysis

TABLE XI
PAIRED DIFFERENCES (HOLM-ADJUSTED WILCOXON)

Comparison	Δ Throughput	Δ Power	Δ Efficiency	Δ p95 latency (ms)
Perf+IPC vs Best-Static	-0.043 (d _z =-2.3) *	-0.13 (d _z =-1.7) *	-0.032 (d _z =-2.1) *	+1.07 (d _z =0.3) ns
Perf+IPC vs Heuristic	+0.263 (d _z =16.1) *	+0.65 (d _z =10.8) *	+0.268 (d _z =20.1) *	+11.15 (d _z =4.3) *
Perf+IPC vs Random	+0.096 (d _z =4.5) *	+0.40 (d _z =3.4) *	+0.053 (d _z =2.6) *	-2.39 (d _z =-0.9) *
Perf vs Orig	+0.170 (d _z =1.9) *	+1.56 (d _z =3.0) *	-0.158 (d _z =-2.2) *	-3.23 (d _z =-3.3) *
Perf+IPC vs Perf	+0.069 (d _z =1.3) *	+0.30 (d _z =1.7) *	+0.034 (d _z =0.7)	-1.36 (d _z =-0.5) *

Table XI lists the paired differences (*** $p < 0.001$** , $p < 0.01$, * $p < 0.05$, ns = not significant after Holm correction). All throughput, power and efficiency differences are significant. The IPC feature gives a significant gain over the telemetry-only bandit, and the corrected reward significantly improves throughput over the original reward. The only latency difference that is not significant is Perf+IPC versus Best-Static (+1.07 ms). Because the seeds vary only the workload phases and measurement noise within one fixed model, these tests quantify consistency inside the simulator and should not be read as evidence about real hardware.

5.5 Latency, Fairness and Migration

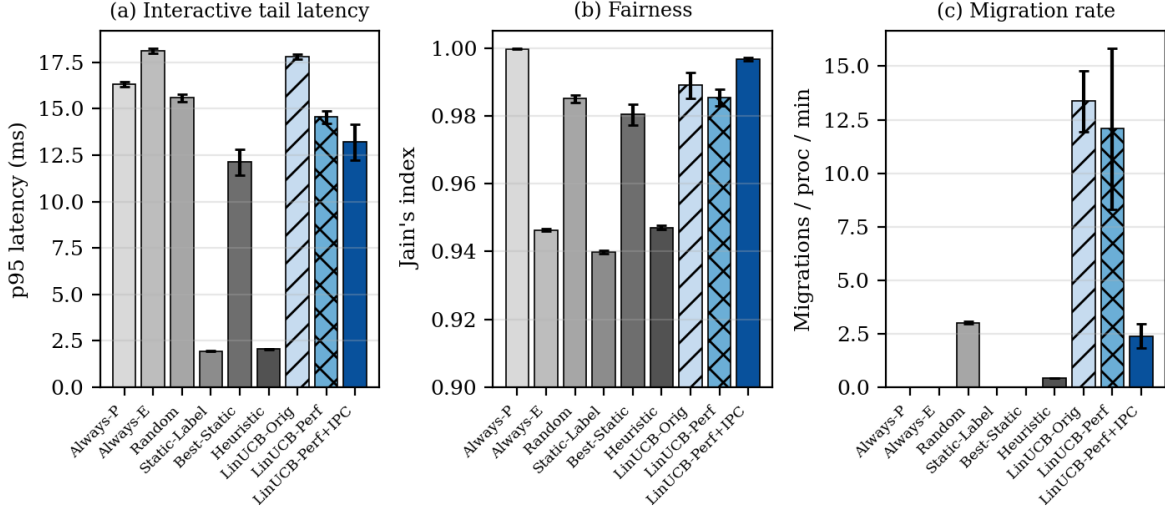


Fig. 13. Interactive tail latency (a), Jain's fairness index (b) and migration rate (c) of the nine policies.

Fig. 13 shows tail latency, fairness and migration rate. Fairness is highest for LinUCB-Perf+IPC among the policies that use both core groups (Jain 0.997). The learned policies have a higher tail latency for the interactive I/O processes (13.2 ms for Perf+IPC; 17.8 ms for Orig) than Static-Label and Heuristic (about 2 ms). The reason is the same as for their low throughput: those two rules park the interactive processes on a nearly empty E-group, whereas the learned policies use all eight cores and therefore share cores with heavier processes. The reward (6) contains no latency term and the interactive processes have small demand, so their latency has little influence on learning. This is a real limitation and the main motivation for the latency-aware reward proposed in Section 6.

5.6 Sensitivity and Overhead

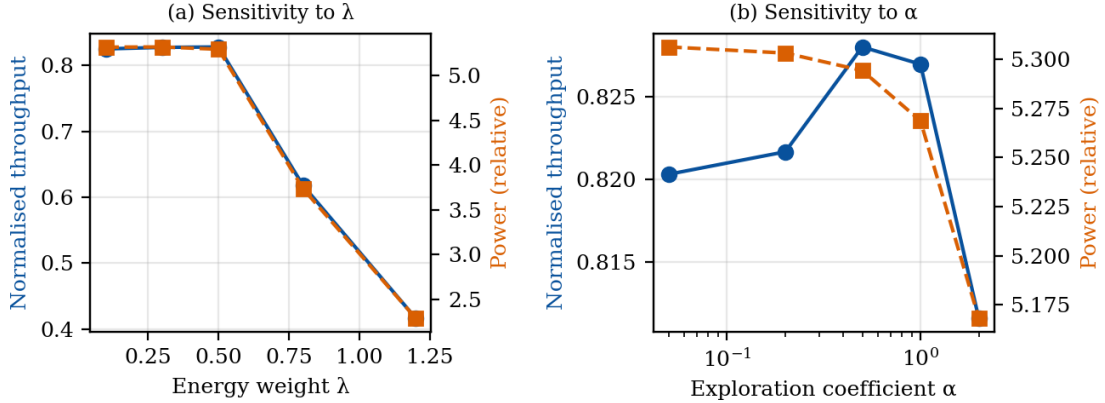


Fig. 14. Sensitivity of LinUCB-Perf+IPC (15 seeds per point) to the energy weight λ (a) and the exploration coefficient α (b).

Fig. 14(a) shows that λ acts as a power–performance knob: for $\lambda \leq 0.5$ throughput stays at 0.825–0.828 with power about 5.3, while $\lambda = 0.8$ gives 0.618 throughput at 3.74 power and $\lambda = 1.2$ gives 0.417 at 2.29, so an operator can move along the frontier without retraining the method. Fig. 14(b) shows that the policy is robust to the exploration coefficient: throughput stays between 0.812 and 0.828 for α from 0.05 to 2.0 and is highest at $\alpha = 0.5$, the default.

TABLE XII
COMPUTATIONAL COST OF ONE BANDIT UPDATE ($D = 6$)

Method	Time per update (μ s)	Complexity
Sherman–Morrison (used)	8.74	$O(d^2)$
Explicit matrix inversion	7.95	$O(d^3)$

Table XII reports the measured cost of one update on a single core. At $d = 6$ the two methods are comparable (the $O(d^2)$ advantage appears only for larger feature vectors), and both are several orders of magnitude below the one-second control period, so the learning overhead is negligible for twelve processes.

5.7 Policy Behaviour and Interpretability

Fig. 15 shows the fraction of time each process type spends on the P-group (mean over 10 seeds). The hindsight-optimal Best-Static places the memory-bound processes entirely on the E-group, three of the four CPU-bound processes and 90% of the bursty ones on P, and mostly keeps the I/O-bound ones on E. LinUCB-Perf+IPC reproduces this pattern without any labels: 92% of the CPU-bound and 66% of the bursty placements are on P, memory-bound processes stay on E (1% on P), and I/O-bound processes are split (39% on P) because their reward is nearly indifferent. LinUCB-Perf without IPC cannot separate CPU-bound from memory-bound processes and spreads both (74% and 29% on P). LinUCB-Orig sends nearly everything to E (41% of the CPU-bound on P, the other types at or below 10%), as predicted by the reward analysis of Section 3.8.

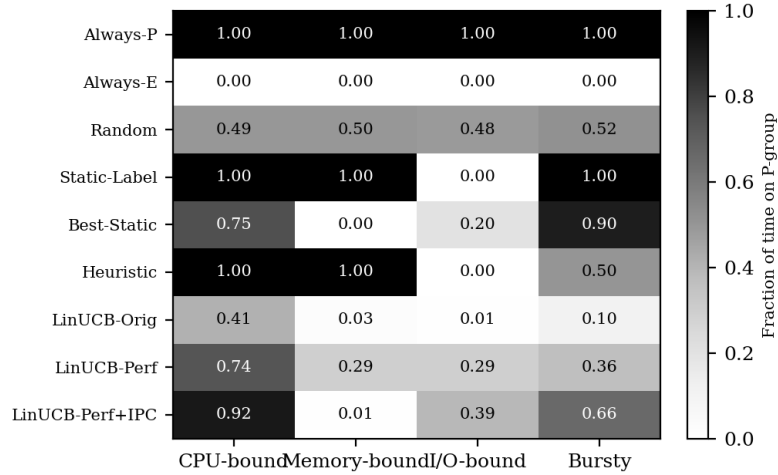


Fig. 15. Fraction of time each process type spends on the P-group under each policy (mean over 10 seeds).

Fig. 16 shows the weights learned by the shared bandit of one seed (features as in Table II). The IPC weight is the clearest signal: it is -0.03 for the P arm but -0.37 for the E arm, so a high IPC lowers the predicted reward of the E-group by about 0.34 relative to P, which is the rule “compute-bound processes need the fast cores”. The intercept of the E arm (0.87) exceeds that of the P arm (0.52), reflecting the energy saving available to processes that tolerate the slower cores. The involuntary-switch weights are almost identical for both arms (-0.31 and -0.31), so contention alone does not favour one group, and the CPU-share weight is negative for E (-0.16) and near zero for P (0.01). These values come from a single seed and serve interpretation, not population estimates.

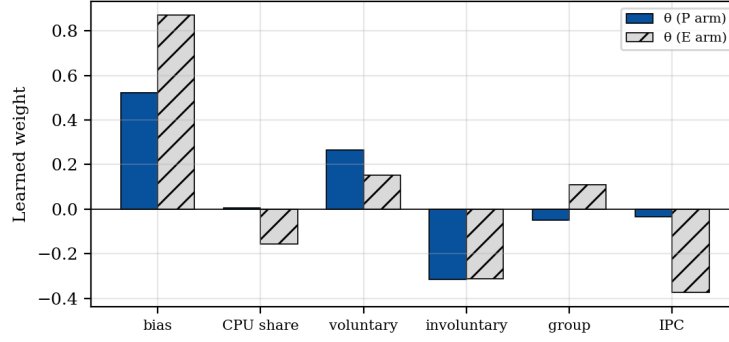


Fig. 16. Weights learned by the shared LinUCB-Perf+IPC bandit for each arm at the end of one 300-cycle episode (seed 0).

5.8 Distribution Across Seeds and Steady State

Fig. 17 shows the spread of throughput and efficiency over the 30 seeds. The 95% confidence half-width of the throughput of LinUCB-Perf+IPC is 0.006, against 0.026 for LinUCB-Orig and 0.020 for LinUCB-Perf, so its advantage is consistent across seeds rather than driven by a few of them. All averages so far include the learning phase. Fig. 18 restricts them to the last 100 cycles: the throughput of LinUCB-Perf+IPC is 0.832 there against 0.827 over all cycles, and the shortfall relative to Best-Static is 4.1% in steady state against 4.9% over the whole episode.

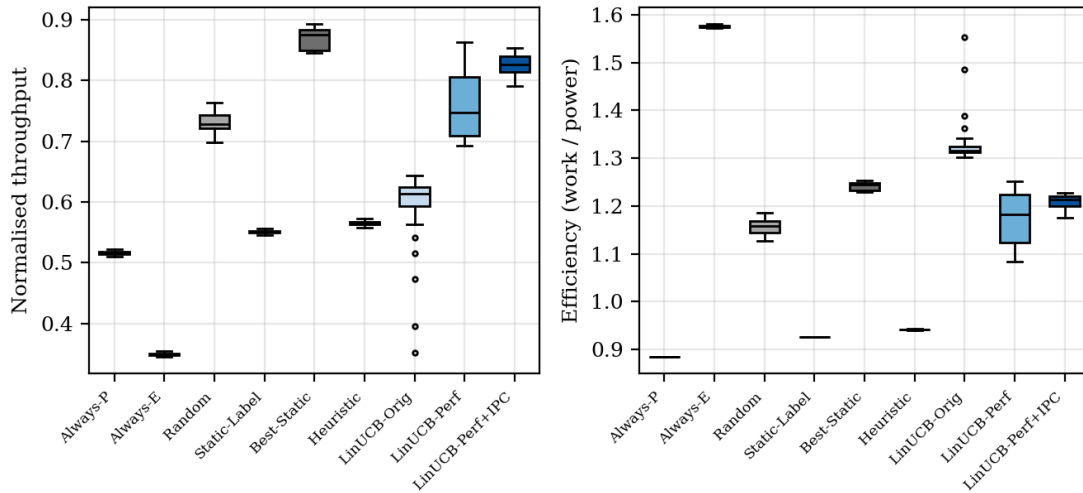


Fig. 17. Distribution over 30 seeds of normalised throughput (a) and efficiency (b); boxes show the interquartile range and whiskers the extremes up to 1.5 times the interquartile range.

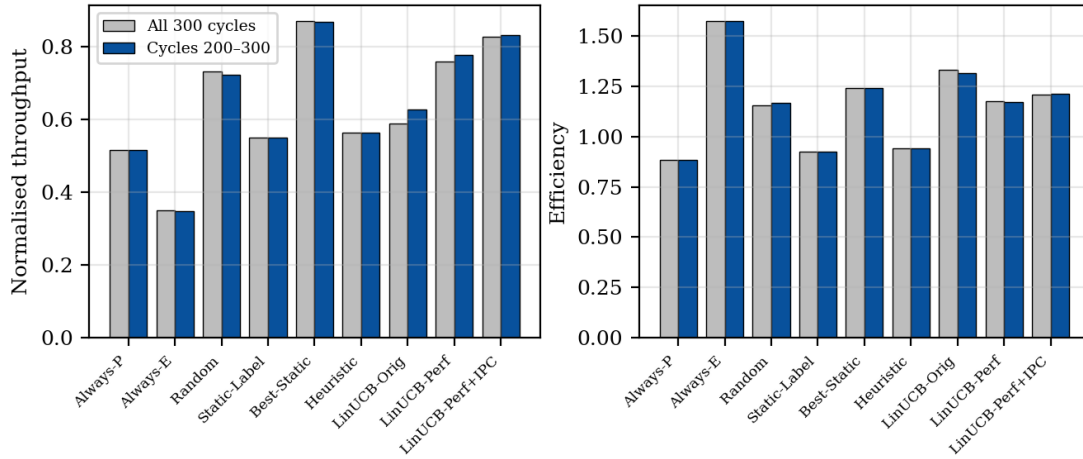


Fig. 18. Normalised throughput (a) and efficiency (b) over all 300 cycles versus the last 100 cycles.

Fig. 19 condenses all metrics. No policy is best in every column: Best-Static leads in throughput, Always-E in power and efficiency, and the label-based and heuristic rules in interactive latency, while the learned policy with IPC combines high throughput with moderate power and a low migration rate.



Fig. 19. Mean value of every metric for every policy; shading indicates relative rank within each column (darker is better, with lower-is-better metrics reversed).

Fig. 20 shows the same information as a radar chart for five policies; each axis is scaled so that the outer ring is the best policy on that metric. The enclosed area is 1.35 for Best-Static, 1.10 for AIZEN with IPC, 0.84 for Random, 0.34 for LinUCB-Orig and 0.24 for the README heuristic; a larger area means a better balance over the seven metrics.

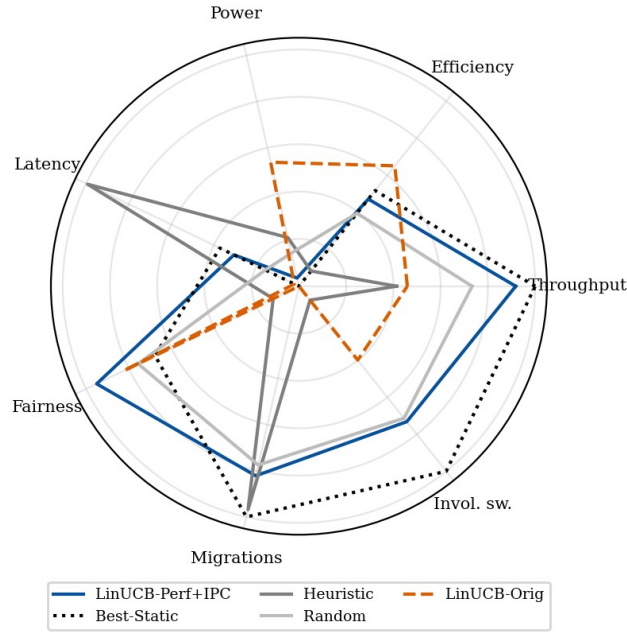


Fig. 20. Radar chart of seven normalised metrics for five policies (outer ring = best policy on that metric).

5.9 Answers to the Research Questions

- **RQ1.** No. With reward (5) the policy behaves as “prefer E”: its throughput is 0.588 against 0.758 with reward (6), a loss of 22%, and only 41% of the CPU-bound processes are placed on P.
- **RQ2.** The IPC-enabled policy reaches 0.827, which is 95% of Best-Static (0.870), 46% above the README heuristic and 13% above random placement.
- **RQ3.** The reward matters most (+29% throughput from (5) to (6)), followed by the IPC feature (+9%). The energy weight λ moves the operating point along the throughput–power frontier, whereas α has only a minor effect.
- **RQ4.** The IPC policy migrates 2.4 times per process per minute (against 12.1 without IPC), its interactive p95 latency is 13.2 ms against about 2 ms for the label-based rules, and one bandit update takes about 9 μ s.

5.10 Discussion and Threats to Validity

- **Simulation, not hardware.** The core speeds, power values, IPC feature and latency model are assumptions. Absolute numbers will differ on a real hybrid CPU; the qualitative findings (reward matters, IPC separates process types, learned policies balance load) need confirmation on Linux hybrid hardware with measured energy.
- **Oracle gap.** Best-Static, which knows the workload in advance, is still 5% better in throughput. The online policy pays for learning and for re-evaluating placements every cycle.
- **Interaction between processes.** One shared bandit makes decisions for processes that compete for the same cores, so the reward of one process depends on the others’ actions; the bandit assumption of independent rounds is only approximate.
- **Metric choice.** The service term uses delivered work divided by demand, which the simulator knows exactly; on a real system it must be estimated from retired-instruction counters or CPU share.
- **Statistics.** Seeds share one model, so significance indicates consistency within the model only.

6. CONCLUSION

This report presented AIZEN, a user-space contextual-bandit governor that chooses between P-core and E-core groups for each process from operating-system telemetry. Auditing the prototype showed that its reward, which combines involuntary context switches with a constant energy proxy, drives the policy toward low power without regard to delivered work, and that the native evaluation artefacts could not support any claim. Replacing the reward with a service-minus-energy signal and adding an IPC feature produced a policy that, in a calibrated 4P+4E simulation with 30 seeds, achieves 0.827 normalised throughput: 46% above the README heuristic, 13% above random placement and 95% of a hindsight-optimal static partition, with about 2.4 migrations per process per minute and a negligible computational cost. The energy weight gives a simple control over the power–performance trade-off.

The main limitations are the simulated platform and the higher tail latency of interactive processes. **Future work** includes (i) re-running the study on Linux with a hybrid CPU, measuring energy with RAPL and IPC with performance counters, and reading run-queue delay from the kernel to add a latency term to the reward; (ii) a corrected native harness that keeps persistent process handles, measures all conditions over identical windows and uses oblivious baselines; (iii) change-point detection such as CUSUM [26] to discount stale statistics when a process changes phase; (iv) a switching-cost penalty and minimum dwell time to avoid thrashing; and (v) comparison with Thompson sampling [27] and with the Linux energy-aware scheduler.

APPENDIX A: KEY CODE LISTINGS

The listings are taken from the simulator (sim.py) that produced the results of Section 5. They show the bandit (Sherman–Morrison form), the two reward functions and the contention model.

A.1 LinUCB with Sherman–Morrison update

```
class LinUCB:
    def __init__(s, d, alpha, rng):
        s.Ai = np.stack([np.eye(d)] * 2) # A-1 of each arm
        s.b = np.zeros((2, d)); s.al = alpha; s.r = rng
    def select(s, x):
        sc = [(s.Ai[a] @ s.b[a]) @ x + s.al * np.sqrt(max(x @ s.Ai[a] @ x, 0))
              for a in (0, 1)]
        if abs(sc[0] - sc[1]) < 1e-12: return int(s.r.integers(2))
        return int(np.argmax(sc))
    def update(s, a, x, r):
        # rank-one update, O(d2)
        Ax = s.Ai[a] @ x
        s.Ai[a] -= np.outer(Ax, Ax) / (1 + x @ Ax)
        s.b[a] += r * x
```

A.2 Reward functions

```
# proposed reward, Eq. (6): served fraction of demand minus attributed energy
sat = np.clip(prog / d * noise, 0, 1.2)
R = sat - lam * cpu * np.where(g == 0, 1.0, 0.30) # lam = 0.5
# original prototype reward, Eq. (5)
R_orig = -(0.7 * np.minimum(invol / 100, 3) + 0.3 * np.where(g == 0, 1.0, 0.3))
```

A.3 Contention model

```
for k, n in ((0, NP), (1, NE)): # P-group, E-group
    m = (g == k); Dg = d[m].sum()
    if Dg > 0: share[m] = d[m] * min(1.0, n / Dg) # Eq. (7)
speed = np.where(g == 0, 1.0, sigma_E); prog = share * speed
power = NP*IDLE_P + NE*IDLE_E + DYN_P*min(NP, D_P) + DYN_E*min(NE, D_E) # Eq. (8)
```

APPENDIX B: REPRODUCTION INSTRUCTIONS

All results can be regenerated on any machine with Python 3 and the packages below. The seeds are fixed, so repeated runs give identical numbers.

```
pip install numpy scipy matplotlib psutil
python sim.py          # about 3 min on one core: results.json, raw_per_seed.csv
python extra.py        # per-type placement and learned weights: extra.json
python figs.py && python figs2.py  # all figures used in this report
```

The files results.json and raw_per_seed.csv contain the per-policy summaries and the per-seed values behind every table and figure, and extra.json contains the data for the placement heat map and the learned weights.

APPENDIX C: SUPPLEMENTARY RESULTS

Table XIII lists the steady-state results (cycles 200–300) and the involuntary-switch rate of every policy.

TABLE XIII
STEADY-STATE AND SUPPLEMENTARY RESULTS

Policy	Throughput (steady)	Power (steady)	Efficiency (steady)	Efficiency (all)	Invol. switches / s
Always-P	0.515 ± 0.003	4.52 ± 0.00	0.885 ± 0.000	0.885 ± 0.000	25.87 ± 0.07
Always-E	0.349 ± 0.002	1.72 ± 0.00	1.574 ± 0.002	1.575 ± 0.001	25.87 ± 0.07
Random	0.723 ± 0.010	4.81 ± 0.08	1.168 ± 0.011	1.157 ± 0.006	10.26 ± 0.30
Static-Label	0.550 ± 0.003	4.61 ± 0.00	0.926 ± 0.000	0.926 ± 0.000	24.79 ± 0.08
Best-Static	0.868 ± 0.008	5.43 ± 0.04	1.242 ± 0.003	1.242 ± 0.003	3.99 ± 0.34
Heuristic	0.563 ± 0.004	4.65 ± 0.00	0.941 ± 0.001	0.942 ± 0.000	24.22 ± 0.09
LinUCB-Orig	0.626 ± 0.024	3.71 ± 0.17	1.316 ± 0.020	1.334 ± 0.020	17.14 ± 0.96
LinUCB-Perf	0.777 ± 0.024	5.14 ± 0.10	1.173 ± 0.020	1.176 ± 0.019	9.93 ± 0.76
LinUCB-Perf+IPC	0.832 ± 0.008	5.33 ± 0.03	1.212 ± 0.006	1.210 ± 0.005	9.85 ± 0.41

REFERENCES

- [1] R. Kumar, K. I. Farkas, N. P. Jouppi, P. Ranganathan, and D. M. Tullsen, “Single-ISA heterogeneous multi-core architectures: The potential for processor power reduction,” in Proc. 36th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO), 2003, pp. 81–92.
- [2] P. Greenhalgh, “big.LITTLE processing with ARM Cortex-A15 & Cortex-A7,” ARM Ltd., White Paper, Sep. 2011.
- [3] C. S. Pabla, “Completely fair scheduler,” Linux Journal, no. 184, Aug. 2009.
- [4] “Energy Aware Scheduling,” Linux Kernel Documentation. [Online]. Available: <https://docs.kernel.org/scheduler/sched-energy.html>
- [5] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, “The Linux scheduler: A decade of wasted cores,” in Proc. 11th Eur. Conf. Computer Systems (EuroSys), 2016, Art. no. 1.
- [6] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
- [7] R. K. Jain, D.-M. W. Chiu, and W. R. Hawe, “A quantitative measure of fairness and discrimination for resource allocation in shared computer systems,” Digital Equipment Corp., Tech. Rep. DEC-TR-301, 1984.
- [8] C. Delimitrou and C. Kozyrakis, “Quasar: Resource-efficient and QoS-aware cluster management,” in Proc. 19th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2014, pp. 127–144.
- [9] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in Proc. ACM SIGCOMM, 2019, pp. 270–288.
- [10] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [11] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, pp. 529–533, 2015.
- [12] P. Auer, “Using confidence bounds for exploitation-exploration trade-offs,” J. Mach. Learn. Res., vol. 3, pp. 397–422, 2002.
- [13] A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” Technometrics, vol. 12, no. 1, pp. 55–67, 1970.
- [14] L. Li, W. Chu, J. Langford, and R. E. Schapire, “A contextual-bandit approach to personalized news article recommendation,” in Proc. 19th Int. Conf. World Wide Web (WWW), 2010, pp. 661–670.
- [15] W. Chu, L. Li, L. Reyzin, and R. E. Schapire, “Contextual bandits with linear payoff functions,” in Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS), 2011, pp. 208–214.
- [16] Y. Abbasi-Yadkori, D. Pál, and C. Szepesvári, “Improved algorithms for linear stochastic bandits,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 24, 2011.
- [17] T. Lattimore and C. Szepesvári, Bandit Algorithms. Cambridge, U.K.: Cambridge Univ. Press, 2020.
- [18] J. Dean and L. A. Barroso, “The tail at scale,” Commun. ACM, vol. 56, no. 2, pp. 74–80, 2013.
- [19] J. Sherman and W. J. Morrison, “Adjustment of an inverse matrix corresponding to a change in one element of a given matrix,” Ann. Math. Statist., vol. 21, no. 1, pp. 124–127, 1950.
- [20] G. Rodola, “psutil: Cross-platform library for process and system monitoring in Python,” GitHub repository. [Online]. Available: <https://github.com/giampaolo/psutil>
- [21] “proc(5): process information pseudo-filesystem,” Linux Programmer’s Manual. [Online]. Available: <https://man7.org/linux/man-pages/man5/proc.5.html>
- [22] Linux man-pages project, “sched_setaffinity(2): set and get a thread’s CPU affinity mask,” Linux Programmer’s Manual. [Online]. Available: https://man7.org/linux/man-pages/man2/sched_setaffinity.2.html
- [23] F. Wilcoxon, “Individual comparisons by ranking methods,” Biometrics Bulletin, vol. 1, no. 6, pp. 80–83, 1945.

- [24] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.
- [25] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ, USA: Lawrence Erlbaum, 1988.
- [26] E. S. Page, “Continuous inspection schemes,” *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.
- [27] W. R. Thompson, “On the likelihood that one unknown probability exceeds another in view of the evidence of two samples,” *Biometrika*, vol. 25, no. 3/4, pp. 285–294, 1933.